

AI Report

We are moderately confident this text is

AI Generated

AI Probability

77%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

laprak OS_Tugas.docx - 10/19/2025

Triawan

LAPORAN

TUGAS SISTEM OPERASI

Dosen Pengampu Mata Kuliah

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

NAMA: HOTIJATUL MUNAUWAROH

NIM: 2410651041

KELAS: B

Tanggal : 6 Oktober 2025

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

TUGAS 1 PROCESS MANAGER

Buat program sederhana yang mengelola process menggunakan konsep forking dan inter process communication (IPC) dasar. Program ini harus mendemonstrasikan pemahaman tentang process creation, synchronization, dan komunikasi.

Persiapan dan kompilasi

Baris perintah `mkdir -p praktikum_os/monitor_dir` untuk membuat direktori awal bernama `praktikum_os/monitor_dir`

`cd praktikum_os/monitor_dir` untuk berpindah ke direktori yang baru dibuat

`nano process_manager.c` untuk membuka editor nano untuk membuat atau mengedit file kode C bernama `process_manager.c`.

kode syntax di GNU NANO:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <string.h>
```

```
#include <time.h>
```

```
// --- Fungsi Tugas ---
```

```
// Child 1: Faktorial
```

```
long long factorial(int n) {
```

```
    if (n == 0) return 1;
```

```
    return (long long)n * factorial(n - 1);
```

```
}
```

```
// Child 2: Bilangan Prima (sederhana)
```

```
void find_primes(int limit, char *output) {
```

```
    strcpy(output, "");
```

```
    for (int i = 2; i <= limit; i++) {
```

```
        int is_prime = 1;
```

```
        for (int j = 2; j * j <= i; j++) {
```

```
            if (i % j == 0) {
```

```
                is_prime = 0;
```

```
                break;
```

```
            }
```

```

}

if (is_prime) {

char temp[10];

sprintf(temp, "%d ", i);

strcat(output, temp);

}

}

}

```

// Child 3: String Terbalik

```

void reverse_string(const char *input, char *output) {

int len = strlen(input);

for (int i = 0; i < len; i++) {

output[i] = input[len - 1 - i];

}

output[len] = '\0';

}

```

// --- Struktur Data untuk Shared Memory Sederhana (digantikan oleh file/pipe di contoh ini untuk kesederhanaan)

// Dalam implementasi nyata, data ini akan ada di Shared Memory (shmget/shmat).

```

void run_child(int child_id, int input_val) {

time_t start_time = time(NULL);

printf("[Child %d] Mulai...\n", child_id);

if (child_id == 1) {

// Simulasi input: Angka untuk Faktorial

printf("[Child 1] Masukkan angka untuk faktorial: %d\n", 5);

long long result = factorial(5);

sleep(2); // Simulasi waktu eksekusi

printf("[Hasil Child 1] Faktorial: %lld\n", result);

} else if (child_id == 2) {

```

```

// Simulasi input: Batas Bilangan Prima

printf("[Child 2] Masukkan batas bilangan prima: %d\n", 10);

char result[20];

find_primes(10, result);

sleep(3); // Simulasi waktu eksekusi

printf("[Hasil Child 2] Bilangan Prima: %s\n", result);

} else if (child_id == 3) {

// Simulasi input: String

const char *input_str = "23";

printf("[Child 3] Masukkan string: %s\n", input_str);

char result[5];

reverse_string(input_str, result);

sleep(4); // Simulasi waktu eksekusi

printf("[Hasil Child 3] String Terbalik: %s\n", result);

}

time_t end_time = time(NULL);

printf("[Child %d] Selesai. Waktu: %.2f detik\n", child_id, difftime(end_time, start_time));

exit(0);

}

int main() {

printf("=== Tugas 1: Process Manager (NIM: 23, varian GANJIL / Shared Memory) ===\n");

pid_t pids[3];

int status;

clock_t start_time, end_time;

// --- Membuat dan Menjalankan Child 1 ---

start_time = clock();

pids[0] = fork();

if (pids[0] == 0) { run_child(1, 5); } // Child

```

```

else if (pids[0] < 0) { perror("fork 1 error"); exit(1); }

// --- Membuat dan Menjalankan Child 2 ---

pids[1] = fork();

if (pids[1] == 0) { run_child(2, 10); } // Child

else if (pids[1] < 0) { perror("fork 2 error"); exit(1); }

// --- Membuat dan Menjalankan Child 3 ---

pids[2] = fork();

if (pids[2] == 0) { run_child(3, 23); } // Child

else if (pids[2] < 0) { perror("fork 3 error"); exit(1); }

// --- Parent Menunggu dan Melaporkan Hasil (Berurutan) ---

for (int i = 0; i < 3; i++) {

// waitpid: tunggu child spesifik selesai

pid_t child_pid = waitpid(pids[i], &status, 0);

end_time = clock();

double time_taken = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;

printf("\n[Parent] Child selesai. PID=%d | Exit=%d | Time=%.6f detik\n",

child_pid, WEXITSTATUS(status), time_taken);

// Dalam implementasi sebenarnya, di sini Parent akan membaca data dari Shared Memory

// Untuk contoh ini, output hasil ada di dalam fungsi Child.

}

printf("\n=== Semua proses selesai ===\n");

return 0;

}

```

gcc process_manager.c -o process_manager untuk kompilasi file sumber C (process_manager.c) menggunakan GCC (GNU Compiler Collection), dan menghasilkan file executable bernama process_manager.

Peringatan kompilator

process_manager.c: In function 'find_primes':

process_manager.c:28:30: warning: '%' directive writing 1 byte into a region of size between 0 and 9 [-Wformat-overflow]

```
28 | sprintf(temp, "%d ", i);
```

```
| ^
```

process_manager.c:28:13: note: 'sprintf' output between 3 and 12 bytes into a destination of size 10

```
28 | sprintf(temp, "%d ", i);
```

```
| ^~~~~~
```

. Analisis Peringatan:

Fungsi: Peringatan terjadi di dalam fungsi bernama find_primes.

Baris: Baris ke-28 adalah sprintf(temp, "%d ", i);.

Masalah: Kompilator GCC (dengan opsi seperti -Wformat-overflow) memperingatkan adanya potensi buffer overflow.

Detail: Variabel temp (yang digunakan sebagai destination sprintf) memiliki ukuran 10 byte. Output dari sprintf (%d) bisa berkisar antara 3 hingga 12 byte (misalnya, angka 999999999 diikuti spasi adalah 10 byte, ditambah null terminator menjadi 11 byte). Ada kemungkinan output yang dihasilkan melebihi ukuran buffer (10 byte), yang dapat menimpa memori setelah temp.

Dampak: Meskipun program tetap terkompilasi dan berjalan (karena hanya peringatan, bukan error), ini menunjukkan kerentanan yang harus diperbaiki dengan menambah ukuran buffer temp (misalnya, menjadi 16 atau 32 byte untuk amannya).

Eksekusi program

Baris perintah./process_manage untuk Menjalankan file executable yang telah dikompilasi di direktori saat ini.

Keluaran Program (Output):

Inisialisasi Program Utama

=== Tugas 1: Process Manager (NIM: 1, varian GANJIL / Shared Memory) ===

Program memulai dengan mencetak judul dan detail konfigurasi (Tugas 1, NIM: 1, varian GANJIL / Shared Memory). Ini mengindikasikan program ini kemungkinan menggunakan Shared Memory untuk komunikasi antar proses.

Proses Anak (Child Processes)

Proses child 1 tugasnya menghitung factorial input 5

Proses child 2 mencari bilangan prima batas 10

Proses child 3 membalikkan string "1"

3. Kesimpulan

Program process_manager berhasil dikompilasi dan dieksekusi. Program ini mendemonstrasikan konkurensi dengan membagi tiga tugas berbeda (Faktorial, Bilangan Prima, String Terbalik) ke dalam tiga proses anak. Tugas-tugas ini dieksekusi secara paralel/konkuren, dan proses induk memantau penyelesaiannya.

Poin Utama:

Program adalah implementasi sistem operasi (OS) yang melibatkan manajemen proses (melalui `fork()`) dan kemungkinan IPC (Inter-Process Communication) seperti Shared Memory.

Terdapat potensi kerentanan buffer overflow dalam fungsi `find_primes` yang harus diperbaiki.

Semua proses anak berhasil menyelesaikan tugasnya, dengan durasi bervariasi dari 2.00 hingga 4.00 detik per tugas.

Tugas 2: File Monitor

Buat program yang memantau perubahan pada direktori tertentu dan mencatat aktivitas file, menggunakan system calls untuk file operations.

`file_monitor` bertujuan untuk memantau (monitoring) aktivitas file (seperti pembuatan, modifikasi, dan penghapusan) dalam direktori tertentu (`/tmp/monitor_dir`).

Pembuatan direktori:

```
$ mkdir -p praktikum_05/monitor_dir
```

```
$ cd praktikum_05/monitor_dir
```

Pembuatan file sumber:

```
$ nano file_monitor.c
```

Kompilasi program;

```
$ gcc file_monitor.c -o file_monitor
```

Eksekusi program pemantauan:

```
$/file_monitor /tmp/monitor_dir
```

CODE GNU:

```
GNU nano 7.2 file_monitor.c #include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/inotify.h>
```

```
#include <errno.h>
```

```
#include <string.h>
```

```
#include <time.h>
```

```
#include <sys/types.h>
```

```
#include <signal.h>
```

```
#include <fcntl.h> // Diperlukan untuk open()

// Ukuran dasar inotify_event

#define EVENT_SIZE (sizeof(struct inotify_event))

// Ukuran buffer harus cukup besar untuk menampung beberapa event

#define EVENT_BUF_LEN (1024 * (EVENT_SIZE + 16))

// File Descriptors global untuk inotify, watch, dan log

int inotify_fd = -1;

int wd = -1;

int log_fd = -
```

● Sentences that are likely AI-generated.

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 10 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.